

ROS2 센서 데이터 파이프라인 & rosbag2

LiDAR · IMU · Camera 토픽 & 데이터 녹화·재생

cchyun@gmail.com

Part 1. LiDAR 센서 —

`sensor_msgs/LaserScan`

- LiDAR(Light Detection and Ranging)

- 레이저를 발사하여 물체에 반사되어 돌아오는 **시간을 측정**해 거리를 계산하는 센서
- ROS2에서 2D LiDAR 데이터는 `sensor_msgs/msg/LaserScan` 메시지 타입 사용
- TurtleBot3는 `/scan` 토픽을 통해 LiDAR 데이터 발행

- `sensor_msgs/msg/LaserScan` 주요 필드

- `angle_min` / `angle_max`: 스캔 시작·끝 각도 (라디안, 정면 = 0 기준)
- `angle_increment`: 레이저 빔 사이의 각도 간격 (해상도)
- `ranges`: 각 각도에서 측정된 거리값 **배열(Array)**, 단위: 미터(m)
- `intensities`: 반사 신호의 세기 — 물체 재질에 따라 다름
- `range_min` / `range_max`: 센서가 측정 가능한 최소·최대 거리 범위

- `ranges` 배열 구조: `angle_min`에서 시작, `angle_increment`씩 증가하며 채워짐

- 특정 방향의 인덱스 계산법:

$$\text{Index} = \frac{\text{목표 각도(rad)} - \text{angle_min}}{\text{angle_increment}}$$

- 예시: 정면(0도) 데이터를 얻으려면 위 공식으로 계산된 인덱스의 `ranges` 값 참조
- TurtleBot3 Burger 기준: 360개 샘플, 1도 간격

- 측정 범위 밖의 데이터는 특수값으로 표현됨
 - `inf`: `range_max`를 초과하는 거리 (너무 먼 물체)
 - `NaN`: 너무 가깝거나 측정 오류가 발생한 경우
- 처리 방법: Python 코드 작성 시 반드시 필터링 필요

```
import math
# 유효하지 않은 값 필터링
valid_ranges = [r for r in msg.ranges
                 if not math.isinf(r) and not math.isnan(r)]
```

- 필터링하지 않으면 로봇 오작동의 원인이 됨

Part 2. IMU 센서 — `sensor_msgs/Imu`

- IMU(Inertial Measurement Unit)
 - 로봇의 관성 정보를 측정하여 자세와 움직임을 파악하는 센서
 - ROS2 메시지 타입: `sensor_msgs/msg/Imu`
 - 구성 요소
 - 자이로스코프: 회전 속도(각속도) 측정
 - 가속도계: 선형 가속도(중력 포함) 측정

- `sensor_msgs/msg/Imu` 주요 필드
 - `orientation`: 로봇의 현재 자세를 쿼터니언(Quaternion) 형태로 표현
 - `angular_velocity`: 회전 속도(각속도, rad/s) — X, Y, Z축별
 - `linear_acceleration`: 선가속도(m/s^2) — X, Y, Z축별
 - `covariance`: 각 데이터의 측정 불확실성(공분산)
- 단위 주의: IMU 가속도 데이터는 m/s^2 단위 사용

- 자이로스코프 — `angular_velocity`
 - 로봇이 얼마나 빠르게 회전하는지 측정
 - 짧은 시간 동안의 **방향 변화**를 **정밀하게** 파악
- 가속도계 — `linear_acceleration`
 - 중력 가속도를 포함한 선형 움직임 측정
 - 정지 상태에서 로봇의 **기울기(Roll, Pitch)**를 **파악**하는 기준

Part 3. Camera 센서 - Image vs CompressedImage

카메라 메시지 타입 차이

- 카메라 데이터는 대역폭 소모가 크므로 목적에 맞는 메시지 타입 선택이 중요

- `sensor_msgs/Image` — Raw 데이터
 - 압축되지 않은 원본 데이터
 - 화질 손실 없음
 - 네트워크 대역폭 많이 차지 → **로봇 내부 연산용**에 적합
- `sensor_msgs/CompressedImage` — 압축 데이터
 - JPEG/PNG 등으로 압축
 - 대역폭 절약 → **원격 PC 모니터링** 시 유리

- ROS2 이미지 메시지는 OpenCV에서 직접 사용 불가
 - `cv_bridge` 라이브러리를 사용하여 변환 필요
 - 변환 흐름: ROS2 Image → OpenCV Mat (numpy array)

```
from cv_bridge import CvBridge
import cv2

bridge = CvBridge()

def image_callback(self, msg):
    # ROS2 Image → OpenCV
    cv_image = bridge.imgmsg_to_cv2(msg, "bgr8")
    cv2.imshow("Camera", cv_image)
```

- `image_transport` 라이브러리
 - 다양한 압축 형식을 플러그인 방식으로 지원
 - 코드 변경 없이도 대역폭 최적화 가능
 - 장점
 - Raw, compressed, theora 등 다양한 전송 형식 지원
 - 네트워크 환경에 따라 유연하게 전환 가능
 - 구독/발행 코드의 변경 없이 압축 방식 변경 가능

Part 4. TF2 프레임 연동 (Spatial Awareness)

- 로봇 센서 데이터가 어느 부위에서 오는지 정의해야 함
 - 각 메시지의 `header.frame_id` 를 설정하여 센서 위치 지정
 - 센서별 frame_id 설정 예시

센서	frame_id
LiDAR	<code>base_scan</code> (또는 <code>laser_link</code>)
IMU	<code>imu_link</code>
Camera	<code>camera_link</code>

- 정확한 설정이 있어야 RViz2에서 로봇 모델 위에 센서 데이터가 올바른 위치에 시각화됨

Part 5. 센서 토픽 디버깅 및 시각화

- ROS2 토픽 상태 확인 명령어

- `ros2 topic echo /토픽명`
 - 데이터 내용 실시간 출력
 - 예: `ros2 topic echo /scan`
- `ros2 topic hz /토픽명`
 - 데이터가 설정된 주기(Hz)로 잘 들어오는지 확인
- `ros2 topic bw /토픽명`
 - 카메라 토픽 등이 차지하는 대역폭(Bandwidth) 확인

- RViz2 실행 및 센서 데이터 시각화 절차

1. 실행: 터미널에 `rviz2` 입력
2. Fixed Frame 설정: 보통 `odom` 또는 `base_footprint`로 설정
3. 데이터 추가 (Add 버튼):
 - LiDAR: `LaserScan` 선택 후 `/scan` 토픽 지정
 - IMU: `Imu` 디스플레이 추가 (필요 시 플러그인 설치)
 - Camera: `Image` 디스플레이 추가 후 해당 토픽 지정

Part 6. rosbag2 녹화 & 재생

- ROS 2의 `rosvag2`는 로봇 토픽 데이터를 기록하고 재생하는 필수 도구
 - 실물 로봇 없이 반복 테스트 가능
 - 실제 로봇의 센서 데이터(LiDAR, IMU, Camera 등)를 파일로 저장
 - 실물 로봇 없이도 기록된 데이터를 재생하여 알고리즘 반복 테스트·검증 가능
 - 현장 버그 재현
 - 로봇이 현장에서 예상치 못한 동작 시, 당시 데이터를 기록
 - 사무실에서 동일한 상황을 수없이 재현하며 디버깅 가능
 - 재현하기 어려운 간헐적 버그를 잡는 데 매우 효과적

- 전체 vs 선택적 녹화

- 전체 토픽 녹화: 현재 발행 중인 모든 토픽 캡처

```
ros2 bag record -a
```

- 선택적 녹화: 특정 토픽만 기록

```
ros2 bag record /scan /camera/image_raw /imu
```

- 저장 형식: ROS 2 Humble 기준 SQLite3가 기본
- 생성 결과: 데이터베이스 파일 + `metadata.yaml` 을 포함하는 디렉토리

- 대용량 데이터 관리를 위한 분할 옵션

- 최대 파일 크기 제한

```
ros2 bag record -a --max-bag-size 1073741824 # 1GB 단위로 분할
```

- 최대 녹화 시간 제한

```
ros2 bag record -a --max-bag-duration 60 # 60초 단위로 분할
```

- 분할된 bag 파일들은 순서대로 번호가 붙어 자동 관리됨

- 기록된 데이터를 실제 로봇이 작동하는 것처럼 다시 발행

- 기본 재생

```
ros2 bag play <bag_directory_name>
```

- 재생 속도 조절 (`-r` 옵션)

```
ros2 bag play my_bag -r 2.0    # 2배속 재생  
ros2 bag play my_bag -r 0.5    # 0.5배속 재생
```

- 특정 시간 지점부터 재생

```
ros2 bag play my_bag --start-offset 30.0    # 30초 지점부터
```


- 기록된 토픽 이름과 알고리즘이 요구하는 토픽 이름이 다를 때 활용

- 리맵핑 재생 예시

```
# 기록된 /lidar_scan을 /scan으로 이름 바꿔 재생
ros2 bag play my_bag --remap /lidar_scan:=/scan

# 카메라 토픽 리맵핑 + 속도 조절 동시 적용
ros2 bag play my_bag_folder -r 0.5 \
  --remap /camera/image_raw:=/camera/color/image_raw
```

- 언제 필요한가?

- 다른 로봇/환경에서 기록된 bag 파일을 현재 시스템에 맞게 재생할 때

- 기록된 bag 파일의 상세 정보 확인

```
ros2 bag info <bag_directory_name>
```

- 확인 가능한 정보
 - 지속 시간(Duration): 총 녹화 시간
 - 시작 및 종료 시간: 기록 시작/끝 타임스탬프
 - 총 메시지 수: 전체 기록된 메시지 개수
 - 토픽 목록: 포함된 토픽 이름 및 각 토픽의 데이터 타입
 - 직렬화 형식(Serialization format)

- `rosvag2_py` 패키지로 코드 내에서 직접 bag 파일 읽기/쓰기 가능

- 주요 클래스

- `SequentialWriter`: 순차적으로 데이터 기록
- `SequentialReader`: 순차적으로 데이터 읽기

- 작동 방식

- `TopicMetadata` 객체로 토픽 이름·타입·직렬화 형식 정의 및 등록
- 기록 전 `serialize_message()` 로 메시지를 직렬화된 형태로 변환
- 직렬화된 데이터를 타임스탬프와 함께 `writer.write()` 로 기록

- 현대적인 로봇 개발 흐름 (4단계)

1. 녹화(Record)

- 실물 로봇 또는 고충실도 시뮬레이션에서 센서 데이터 기록

2. 알고리즘 개발

- 기록된 데이터를 바탕으로 SLAM·내비게이션 알고리즘 코딩

3. 검증(Validation)

- `rosviz2` 재생으로 알고리즘 동작 여부를 RViz2 등에서 반복 확인

4. 실물 테스트

- bag 데이터로 충분히 검증된 알고리즘을 실물 로봇에 최종 적용

- **rosbag2** 사용 시 주의사항
 - **디렉토리 중복**: **rosbag2** 는 기존 가방 디렉토리를 덮어쓰지 않음
 - 동일 이름으로 재녹화 시 기존 디렉토리 삭제 또는 이름 변경 필요
 - **ROS 버전 불일치**: 버전 간 재생 시 호환성 문제 발생 가능
 - **TF 변환 누락**: 많은 데이터셋에서 **/tf**, **/tf_static** 이 빠져 RViz 시각화 깨짐
 - 녹화 시 반드시 **TF 토픽 포함** 권장
 - **리맵핑 오류**: 알고리즘 요구 토픽 이름 ≠ 기록 이름 → 데이터 미전달

Camera 녹화·재생 시퀀스 예시

1. 필요한 토픽 확인 후 선택적 녹화

```
ros2 bag record /image_raw
```

(녹화 중단: Ctrl + C)

2. bag 파일 내용 확인

```
ros2 bag info rosbag2_2026_05_10-14_33_29
```

3. 재생

```
ros2 bag play rosbag2_2026_05_10-14_33_29
```

4. RViz2 실행 (별도 터미널)

```
rviz2
```

Image(/camera/image_raw) 디스플레이 추가

TurtleBot3 녹화·재생 시퀀스 예시 (1/2)

```
# === TurtleBot ===  
# 1. 런처 실행  
cd Workspace/ros2_ws  
source install/setup.bash  
ros2 launch my_robot_bringup camera_robot.launch.py  
  
# 1. 필요한 토픽 확인 후 선택적 녹화  
ros2 bag record /tf /tf_static /scan /imu /odom /cmd_vel /image_raw/compressed  
# (녹화 중단: Ctrl + C)  
  
# 2. 파일 압축  
tar zcvf rosbag2_2026_05_10-12_58_04.tgz rosbag2_2026_05_10-12_58_04
```



```
# ==== 노트북 ====  
# 3. 파일 다운로드  
scp ubuntu@192.168.10.4:~/rosbag2_2026_05_10-12_58_04.tgz .  
  
# 4. 압축 풀기  
tar zxvf rosbag2_2026_05_10-12_58_04.tgz  
  
# 기존 모든 ros 프로그램 종료 후 실행  
# 5. compressed → raw 변환 (다른 터미널)  
sudo apt install ros-humble-compressed-image-transport  
sudo apt install ros-humble-image-transport-plugins  
ros2 run image_transport republish compressed raw \  
  --ros-args \  
  --remap in/compressed:=/image_raw/compressed \  
  --remap out:=/image_raw  
  
# 6. 재생  
ros2 bag play rosbag2_2026_05_10-12_58_04  
  
# 7. rviz  
sudo apt install ros-humble-rviz-imu-plugin  
rviz2
```